Bioinformatics, YYYY, 0–0
doi: 10.1093/bioinformatics/xxxxx
Advance Access Publication Date: DD Month YYYY
Manuscript Category

Subject Section

Regulator network learning algorithm based on Kernel Conditional Independence Cluster Permutation Test (KCICPT) algorithm

Xun Guo [1, 2], Yi Liu [2, †], Zhengwei Xie [3, †]

[1]Intern, Department of Pharmacology, School of Basic Medical Sciences, Peking University, 38 Xueyuan Lu, Haidian District, Beijing, 100191, China,

[2]Beijing Key Lab of Traffic Data Analysis and Mining, School of Computer and Information Techn ology, Beijing Jiaotong University, Beijing, China，

[3]Department of Pharmacology, School of Basic Medical Sciences, Peking University, 38 Xueyuan Lu, Haidian District, Beijing, 100191, China,

[†]To whom correspondence should be addressed.

Associate Editor: XXXXXXX

Received on XXXXX; revised on XXXXX; accepted on XXXXX

Abstract

Motivation: The causal network is embedded in the observations of variables through status sampling or perturbed measurements. Machine learning is developed for the automated derivation of causal influences in regulatory networks. One of the long-expected goals is to find an algorithm that accurately predicts the regulatory network. In here, Kernel methods improve the prediction accuracy of the regulatory network by mapping data to high dimensional spaces.

Results: In this study, we incorporate the KCICPT(Kernel Conditional Independence Cluster Permutation Test) algorithm with PC algorithm to learn a regulatory network. We developed Incompletely-Cholesky decomposition methods to accelerate the conditional independence tests. Our algorithm is able to learn a network based on high dimensional data. We first demonstrate the concept by applying KCICPT-PC to a simple three-node network. Then we validate its prediction by comparing it with other regulatory network algorithm. Finally we demonstrate its power on a real biological network and obtain reasonable results.

Availability and Implementation: Algorithm was implemented by matlab, and the code will be accessible on github. Sample code was embedded in supplementary text file.

Contact: yiliu@bjtu.edu.cn, xiezhengwei@hsc.pku.edu.cn

Supplementary information: Supplementary data are available at *Bioinformatics* online.

1 Introduction

Detecting conditional independence relationships between random variables is a challenging and important task for problems such as Bayesian network learning and causal discovery (Bergsma, 2004). We use Bayesian network to represent real biological regulatory networks. One major technique for Bayesian network learning is the PC algorithm, which requires high-precision conditional independence (CI) tests. Previous Bayesian network learning mostly employ Fisher test, mutual information or Z test for these process. In general, such methods are not suited for precisely quantifying the conditional independence for continuous and high-dimensional variables. It's reported that kernel methods improve the accuracy of conditional independence test through mapping the training data to a high dimensional space (Zhang, 2011). Besides, permutation-based tests have been successfully applied to conditional independence tests for discrete-variable, but the method is not applicable to continuous variables (Tsamardinos and Borboudakis, 2010). Recently, A new kernel based conditional independent permutation test KCIPT is proposed to handle continuous variables (Gary et al., 2014), but no efficient implementation of this algorithm exists, which prohibit tractable Bayesian network learning due to high computational efficiency.

In this paper, we developed a new algorithm called kernel conditional independence cluster permutation test (KCICPT) in the PC framework. First, we use kernel-based maximum mean discrepancy (MMD) to solve the two-sample test in KCICPT (Gretton et al., 2012a). Then, we use clustering method to improve the efficiency of algorithm. What's more, we use incomplete Cholesky decomposition to greatly reduce the computational complexity of the algorithm.

Our Algorithm KCICPT improves the accuracy of conditional independence test with kernel methods. We demonstrate the application of KCICPT in both a toy model (Fukumizu, 2008) and a full kinase network (Karen Sachs., 2005).

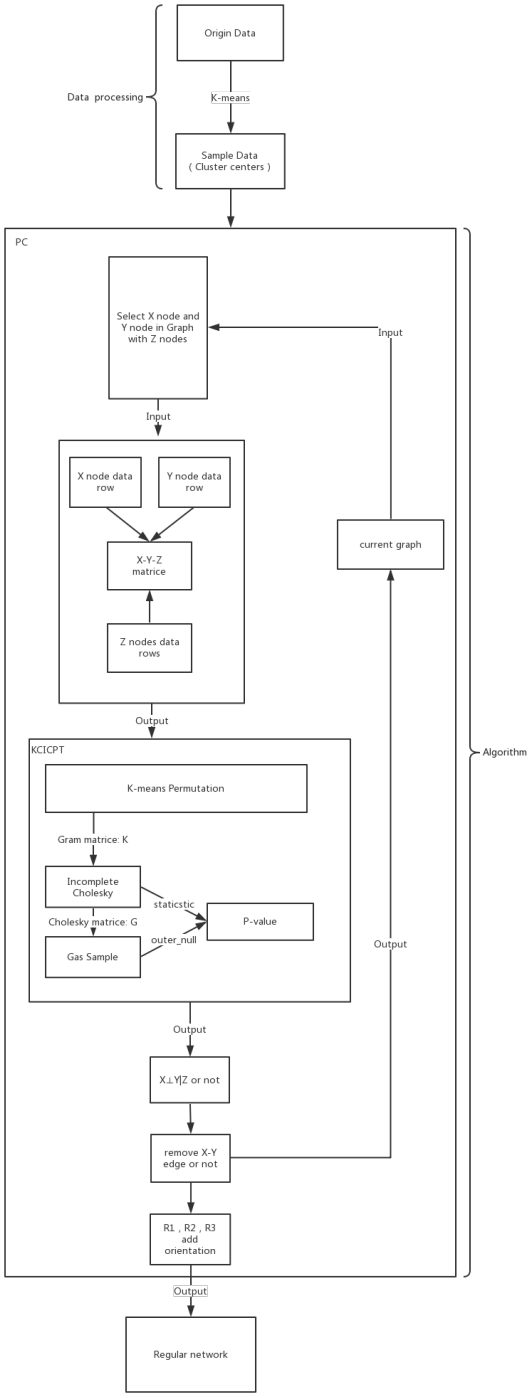


Figure 1. This figure shows the proposed approach of KCICPT-PC. It consists of Data processing and Algorithm, and the algorithm includes PC and KCICPT.

2 Methods

2.1 Basic methods of Conditional Independence Test

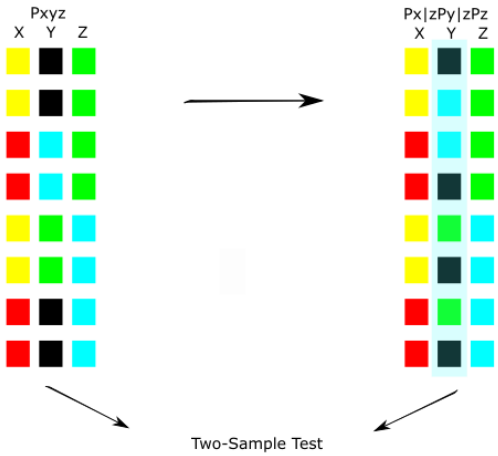


Figure 2. This figure shows the proposed the permutation based conditional independence test. First, we get a sample from the joint distribution (left), and permute the sample to simulate a sample from the factorized distribution. The permutation is chosen to induce conditional independence while preserving $P_{x|z}$, $P_{y|z}$, and P_z. Then, a two-sample test is used to compare the permuted sample to an independent, unpermuted sample from the joint distribution.

As shown in Figure 1, we are interested in determining whether $X \perp Y|Z$ or not, which occurs if and only if $P_{xyz} = P_{x|z}P_{y|z}P_z$. We calculate this conditional independence by comparing permutated statistical steady samples to all permuted samples. To obtain permuted statistical steady samples, we solve a convex optimization problem on conditional variables of conditional independence. Accordingly, we relax the problem to finding a permutation that enforces z $\approx$ P*z (Gary et al., 2014).

Relaxing P to the set of doubly stochastic matrices (matrices whose rows and columns sum to one) with zero trace, the feasible region becomes the convex hull of permutation matrices (Gary et al., 2014), according to Birkhoff-von Neumann theorem (Birkhoff, 1946: von Neumann, 1953).

In this optimization, we have an option to use k-means algorithm to reduce the worst case complexity $O(N^3)$ to $O(n*m^3)$, where m and n is a number far less than N (m<<N, n<<N). Then, we use the kernelization of the sample to compute the test statistic. The original sample Ω of n elements is randomly splitted into half to form the samples $\Omega^{(1)}$ and $\Omega^{(2)}$, we compute the biased MMD test statistic as follows:

$$\mathrm{MMD}\left(\Omega^{(1)}, \Omega^{(2)}\right) = \frac{4}{n^2}\mathbf{1}^T\left(K^{(1)} + K^{(2)} - 2K^{(12)}\right)\mathbf{1} \tag{2}$$

Herein, we use incomplete Cholesky decomposition to reform MMD formula and the Monte Carlo procedure. We successfully reduced the computational complexity of MMD from $O(N^2)$ to $O(m^2*N)$, where m is a number far less than N (m<<N).

2.2 K-means cluster

When we have big samples, we cannot apply the algorithm directly because of the vast time consumption. As previously mentioned, the simplex algorithm has $O(N^3)$ in the worst case. In order to ensure the effect with less time, we use k-means clustering to reduce the running time. Through the k-means clustering algorithm, we find a number of clustering centers. We just permuted inside each cluster and merged these clusters. This operation can improve the speed of convex problem, reduce the worst case complexity from $O(N^3)$ to $O(n*m^3)$, where m is clustering number far less than N (m<<N).

We use this method in Medical Data (Fukumizu et al., 2008) to get the learned Bayesian network which is the same as the algorithm without k-means cluster permutation.

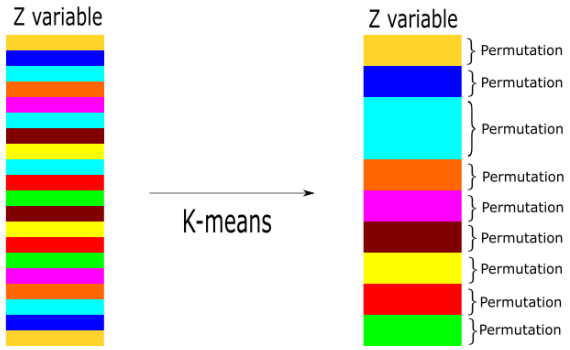


Figure 3. This figure shows how to use k-means to improve the speed of permutation. In the conditional case, X and Y just need to be shuffled when the corresponding Z values are near each other. Therefore, the values of Z are partitioned using a clustering algorithm, where we permute Z values only within clusters.

2.3 Kernel Method and Cholesky Decomposition

In order to improve the power of conditional independence, we use kernel method to build the formula MMD by incomplete Cholesky Decomposition. In a two-sample test, a permuted and unpermuted sample can be used to test the null hypothesis of conditional independence. We use a well-studied kernel based two-sample test by MMD (Gretton et al., 2012a).

In kernel methods, the RBF kernel is used for this test. There is still a problem to be solved, how to select the bandwidth parameter. We set σ to be the median pairwise distance between sample elements (Gretton et al., 2006, 2009, 2012a).

As defined above, we have the algorithm KCICPT. This algorithm consists of two parts, which are outer bootstrap and Monte Carlo iteration. The test statistic relies on splitting a sample randomly into halves, which reduces the power of the two-sample test. If we randomly split the sample many times to compute many test statistics, we can bootstrap the MMD statistic itself to recover some of the power lost due to splitting. The KCICPT is given in Algorithm 1.

Algorithm 1 KCICPT: Kernel Conditional Independence Cluster Permutation Test

Input：Sample$\Omega = \{x, y, z\}$、Distortion δ、Significant level α、Outer Bootstrap iterations B、Inner bootstrap iterations b、Monte Carlo iterations M

Output: Conditional Independence or No Conditional Independence

```
for Outer Bootstrap 1≤i≤ B do
    Split sample into Ω^(1)、Ω^(2)
    Find clusters C for Ω^(2)
    Find permutation matrices Ps for Ω^(2) using δ to compute D and solving (1)
        Get P by cohering Ps
        MMD[i] ← MMD(Ω^(1)、PΩ^(2)) By Cholesky
    for Inner Bootstrap 1≤j≤ b do
        Shuffle, re-split Ω^(1)、PΩ^(2) to Ω′、Ω″
        inner_null[i,j] ← MMD(Ω′、Ω″) By Gaussian Sampling
    end for
end for
statistic ← mean1≤i≤B(MMD[i])
for Monte Carlo Iteration 1≤k≤M do
    for Outer Bootstrap 1≤i≤ B do
        r ← random_interger(1,b)
        samples[i] ← inner_null[i,r]
    end for
    outer_null[k] ← mean1≤i≤B(samples[i])
end for
p-value ← 1 – percentile(statistic, outer_null)
if p-value ≥ α then
    Fail to Reject H0 X independent Y in Z
```

```
else Reject H0  X not independent Y in Z
end if
```

When we handle kernelized MMD, we cannot calculate the formula (2) explicitly because it's time-consuming. We only need to calculate the formula after matrix decomposition. With the incomplete Cholesky decomposition trick, we can avoid an $O(N^2)$ operations in kernel matrices (Gram matrices) (Bach et al. 2001). Herein, we aim to find low-rank approximations of Gram matrices of rank M<<N. We find the fact that positive-semidefinite Gram matrices is a rather strong constraint, allowing approximations to Gram matrices to be found in $O(M^2N)$ operations.

We now show how to use incomplete Cholesky decomposition to solve the MMD and Monte Carlo Iteration.

As aforementioned, we cannot directly compute MMD with a Gram matrix. We can use samples to construct a big Gram matrix, and then use this matrix to apply incomplete Cholesky algorithm. We define vector L_1 from the upper half of the G matrix, L_2 from the lower half of the G matrix.

$$L_1 = (\sum_{j=1}^{N/2} G_{j\,1}, \sum_{j=1}^{N/2} G_{j\,2}, \cdots, \sum_{j=1}^{N/2} G_{j\,M}) \tag{5}$$

$$L_2 = (\sum_{j=N/2}^{N} G_{j\,1}, \sum_{j=N/2}^{N} G_{j\,2}, \cdots, \sum_{j=N/2}^{N} G_{j\,M} \tag{6}$$

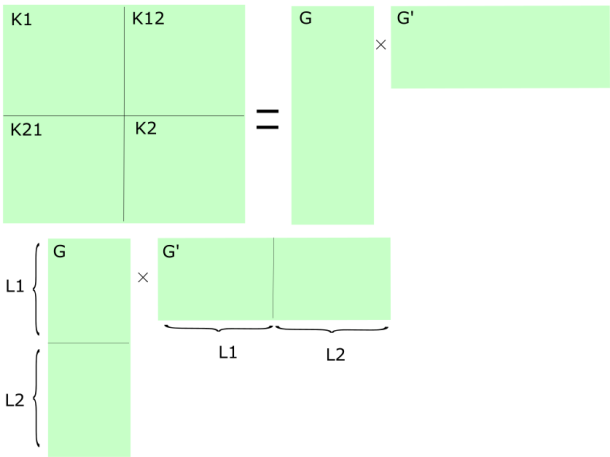


Figure 4. This figure shows a Gram matrix from a concatenated sample which consists of two samples. They are constructed to conduct two-sample test.

We can get a new MMD through incomplete Cholesky operation, and the time complexity of the MMD is $O(m^2N)$.

$$\mathrm{MMD} = L_1 \cdot L_1 + L_2 \cdot L_2 - 2L_1 \cdot L_2 \tag{7}$$

Using this operation, we improve the speed of KCICPT significantly.

2.4 Gaussian Sampling

On the other hand, we can use Gaussian sampling to further improve the efficiency of Monte Carlo iteration in big samples. When we use incomplete Cholesky operation to compute MMD, we need 10000's iterations to obtain the statistic test. We didn't directly compute Monte Carlo iteration each time. We just performed hundreds of iterations, and then used these results to build a Gaussian distribution. We use it to avoid 99% iterative computation. This method greatly improves our algorithm speed in high dimensional data.

3 Result

3.1 Algorithm analysis

Our algorithm uses these methods to improve KCICPT speed, through these operations. We find that the Gaussian Sampling's computing optimization cannot show its advantage in small datasets. However, it can improve the speed in big datasets.

Table 1. This is a result of Medical Data Time consumption analysis, but Gaussian Sampling is not good in this data.

Algorithm	Medical data Time(s)	Power
Origin	25.657297	+
Cholesky	17.675634	+
Cholesky+cluster5	39.150424	+
Cholesky+cluster5+Gaussian	28.787016	-

Table 2. This is a result of Sachs Data Time consumption analysis, and the first result 420000 comes from k-means 1000, because the time of k-means 2000 is unacceptable.

Algorithm	Sachs data k-means2000(s)	Power
Origin	>420000	+
Cholesky	176491.269	+
Cholesky+cluster400	111059.720	+
Cholesky+cluster400+ Gaussian	66695.385139	+

3.2 Medical data

We explored the application of our algorithm to a real-world dataset used in prior work (Fukumizu et al., 2008). The data consists of three variables, creatinine clearance (C), digoxin clearance (D), and urine flow (U), measured on 35 patients. The ground truth, that D is independent of U in condition of C， is known for this dataset. We applied our algorithm with PC and KCICPT with α=0.05 used as a test for conditional independence (Gary et al., 2013). Our algorithm can get a correct result from this data with better performance than original KCICPT. The result is Markov equivalence class D-C-U, which contains the causal structures (either D←C←U, D→C→U, or D←C→U) consistent with the ground truth conditional independence relationship and pairwise dependence relationship.

3.3 Multi-parameter single-cell data

We explored the potential of our algorithm to a real-world dataset, which is multi-parameter single-cell Data (Karen et al., 2005). It consists of flow cytometry measurements of 11 phosphorylated proteins and phospholipids. We call it Sachs data later on. KCICPT-PC with α=0.05 performs better have than other IC-algorithms when adopting K-means method (Nagarajan et al., 2013). The other algorithms consist Grow-Shrink (gs) (Margaritis, 2003), Incremental Association (iamb) (Tsamardinos et al., 2003), Fast Incremental Association (Fast-IAMB) (Yaramakala and Margaritis, 2005), Interleaved Incremental Association (Inter-IAMB) (Tsamardinos et al., 2003). In k-means 6000, we find that our algorithm gets the directed edge Plc→PKC missing in Karen Sachs (2005).

Table 1. This table shows correct directed edges based on the data from Sachs 2005. Conditional independence test: Pearson's Correlation.

cluster-center	GS	iamb	Fast.iamb	Inter.iamb	KCICPT
k-means 2000	3	3	3	3	4
k-means 3000	2	1	3	4	4
k-means 4000	2	1	4	1	4
k-means 5000	1	4	4	4	6
k-means 6000	3	4	4	4	5

Table 2. This table shows correct directed edges based on the data from Sachs 2005. Conditional independence test: Fisher's Z test.

cluster-center	GS	iamb	Fast.iamb	Inter.iamb	KCICPT
k-means 2000	3	3	3	3	4
k-means 3000	2	1	3	4	4
k-means 4000	2	1	4	1	4
k-means 5000	1	4	4	4	6
k-means 6000	3	4	4	4	5

Table 3. This table shows correct directed edges based on the data from Sachs 2005. Conditional independence test: mc-cor in B=1000.

cluster-center	GS	iamb	Fast.iamb	Inter.iamb	KCICPT
k-means 2000	3	3	4	3	4
k-means 3000	1	2	2	3	4
k-means 4000	4	2	6	1	4
k-means 5000	1	3	4	3	6
k-means 6000	3	3	2	4	5

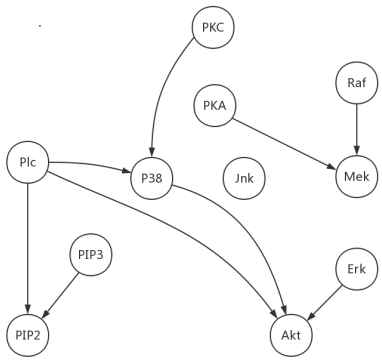


Figure 5. This figure shows a regulatory network from Sachs data by KCICPT in k-means 5000. It recovered sub-network of actual network from the data.

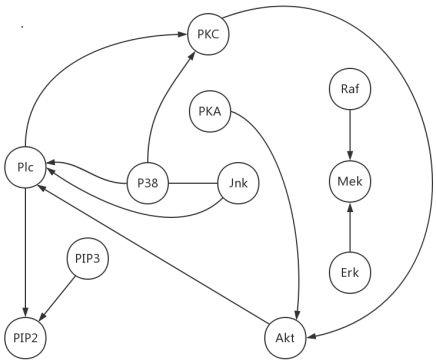


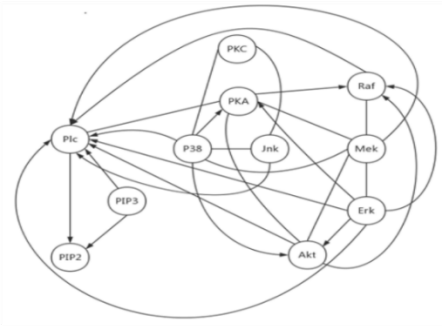
Figure 6. This figure shows a regulatory network from Sachs data by KCICPT in k-means 6000. It recovered sub-network from the data. This result includes Plc →PKC, which is a directed edge missing in Karen Sachs (2005).

As shown in Figure 5, KCICPT can recover more correct directed edges such as Plc→PIP2, PIP3→PIP2, PKA→Mek, Raf →Mek PKC→P38, Erk→Akt in k-means data. The influence path of them is in table 4 (Karen Sachs., 2005). The result from GS, we can find there are some unexplained or wrong edges, such as Plc-Raf, Plc→PKA, Plc→Jnk. The result from iamb, there are Erk→Plc, P38→Akt, Mek→Plc. The result from Fast.iamb, there are Erk→Plc, Raf→Plc, P38→Akt, Mek→Plc. The result from Intert.iamb, there are Erk→Raf, Raf→Plc, P38→Akt, P38→Plc, Mek→Plc.

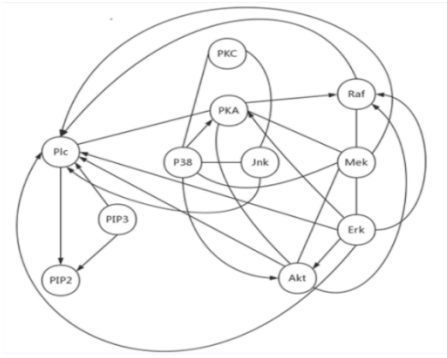
Table 4. This table shows possible molecular pathway of influence represented by arcs in KCICPT in k-means 5000.

Connection	Influence path	Type
Plc→PIP2	Direct hydrolysis to IP3	Direct
PIP3→PIP2	Precursor-product	Direct
PKA→Mek	PKA→Raf_{S621}→Mek	Direct
Raf→Mek	Direct phosphorylation	Direct
PKC→P38	PKC→→MKKs→P38	Direct
Erk→Akt	Direct or Indirect	Direct

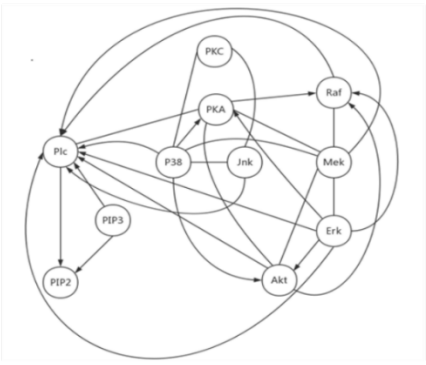
Grow-Shrink(gs)



iamb



Fast-iamb



Inter.iamb

Figure 6. This figure shows a regulatory network from Sachs data by k-means 5000 from GS, iamb, Fast.iamb and Inter.iamb. Conditional independence test is Pearson's Correlation.

4 Conclusion

In this work, we propose a PC algorithm based on a new conditional independence test KCICPT. This algorithm transforms the conditional independence test into two-sample test problem, which is easier to be solved, and is scalable to high dimensional datasets. Through a kernel-based two sample test between an original sample and a permuted sample, which share the same distribution if and only if the conditional independence relationship holds (Gary et al., 2014). We use Cluster Permutation, Gaussian Sampling and Cholesky Decomposition to improve the performance of KCICPT. In general, our algorithm will be increasingly important with the increasing availability and the improved quality of regulatory network learning. In continuous case, KCICPT-PC has better results than other IC algorithms.

Reference

Doran, G., Muandet, K., Zhang, K., & Schölkopf, B. (2014, July). A Permutation-Based Kernel Conditional Independence Test. In *UAI* (pp. 132-141).

Gretton, A., Borgwardt, K. M., Rasch, M., Schölkopf, B., & Smola, A. J. (2006). A kernel method for the two-sample-problem. In *Advances in neural information processing systems* (pp. 513-520).

Gretton, A., Fukumizu, K., Harchaoui, Z., & Sriperumbudur, B. K. (2009). A fast, consistent kernel two-sample test. In Advances in neural information processing systems (pp. 673-681).

Gretton, A., Borgwardt, K. M., Rasch, M. J., Schölkopf, B., & Smola, A. (2012). A kernel two-sample test. *Journal of Machine Learning Research*,*13*(Mar), 723-773.

Sriperumbudur, B. K., Gretton, A., Fukumizu, K., Schölkopf, B., & Lanckriet, G. R. (2010). Hilbert space embeddings and metrics on probability measures. *Journal of Machine Learning Research*, *11*(Apr), 1517-1561.

Fukumizu, K., Gretton, A., Sun, X., & Schölkopf, B. (2008). Kernel measures of conditional dependence. In *Advances in neural information processing systems* (pp. 489-496).

Friedman, N., Nachman, I., & Peér, D. (1999, July). Learning bayesian network structure from massive datasets: the «sparse candidate «algorithm. In *Proceedings of the Fifteenth conference on Uncertainty in artificial intelligence* (pp. 206-215). Morgan Kaufmann Publishers Inc..

Glymour, C., Scheines, R., & Spirtes, P. (2001). *Causation, prediction, and search*. MIT Press.

Bach, F. R., & Jordan, M. I. (2002). Kernel independent component analysis.*Journal of machine learning research*, *3*(Jul), 1-48.

Bach, F. R., & Jordan, M. I. (2003). Learning graphical models with Mercer kernels. In *Advances in Neural Information Processing Systems* (pp. 1033-1040).

Bergsma, W. P. (2004). Testing conditional independence for continuous random variables. Eurandom.

Sachs, K., Perez, O., Pe'er, D., Lauffenburger, D. A., & Nolan, G. P. (2005). Causal protein-signaling networks derived from multiparameter single-cell data. *Science*, *308*(5721), 523-529.

Nagarajan, R., Scutari, M., & Lèbre, S. (2013). Bayesian networks in R.*Springer*, *122*, 125-127.

Schölkopf, B., & Smola, A. J. (2001). *Learning with kernels: support vector machines, regularization, optimization, and beyond*. MIT press.

Meek, C. (1995, August). Causal inference and causal explanation with background knowledge. In Proceedings of the Eleventh conference on Uncertainty in artificial intelligence (pp. 403-410). Morgan Kaufmann Publishers Inc..

Chickering, D. M. (1995, August). A transformational characterization of equivalent Bayesian network structures. In Proceedings of the Eleventh conference on Uncertainty in artificial intelligence (pp. 87-98). Morgan Kaufmann Publishers Inc..

Von Neumann, J. (1953). A certain zero-sum two-person game equivalent to the optimal assignment problem. *Contributions to the Theory of Games*, *2*, 5-12.

Birkhoff, G. (1946). Three observations on linear algebra. *Univ. Nac. Tucumán. Revista A*, *5*, 147-151.

Margaritis, D. (2003). *Learning Bayesian network model structure from data*(Doctoral dissertation, US Army).

Tsamardinos, I., Aliferis, C. F., Statnikov, A. R., & Statnikov, E. (2003, May). Algorithms for Large Scale Markov Blanket Discovery. In *FLAIRS conference*(Vol. 2, pp. 376-380).

Tsamardinos, I., & Borboudakis, G. (2010). Permutation testing improves Bayesian network learning. Machine Learning and Knowledge Discovery in Databases, 322-337.

Yaramakala, S., & Margaritis, D. (2005, November). Speculative Markov blanket discovery for optimal feature selection. In *Data mining, fifth IEEE international conference on* (pp. 4-pp). IEEE.

Zhang, K., Peters, J., Janzing, D., & Schölkopf, B. (2011). Kernel-based Conditional Independence Test and Application in Causal Discovery.

Supplementary

1.1 Network inference with CI test

As with related algorithms that use graphical models to recover regulatory networks (Friedman et al., 1999), we assume that regulatory relationships among variables can be divided into directed and indirected relationships. Assuming a case where all relevant network variables have been measured, conditional independence testing provides a theoretically sound approach for disentangling these relations. Our algorithm framework is PC algorithm (Spirtes et al., 2001). We use PC algorithm to recover the regulatory network skeleton and determine some directions. As shown in Fig. 2, PC algorithm can get directions by these ways. We use V-structure to add direction, set direction of arcs by following two rules (Meek, 1995), and then we get the PDAG (Chickering, 1995).

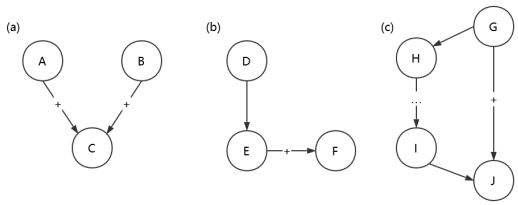


Figure 2. This figure shows the rules of setting direction. (a) is the rule of V structure, when we get indirected graph from PC algorithm. If C node is not the condition of the independence in A and B, we can use this rule to add directions. (b) and (c) are the rules when we get partially directed acyclic graph after (a). The rule (b) sets the direction of E-F to E→F, when D, E has directed edge from D to E. The rule (c) sets the direction between G and J, and there is a strictly directed path from G to J (a path leading from G to J containing no indirected arcs) and then we set the direction of G-J to G→J.

With the properties of direct/indirect relations, the graph of directed interactions can be recovered from observational data. When we have the dependencies and conditional independence mined by the observed data, we can use our conditional independence inference algorithm KCICPT to construct the graph by connecting all correlated pairs of variables, unless there are conditional independences from unobserved variables.

1.2 PC

Our main framework is PC algorithm, which is an important causal inference method.

We use PC algorithm to learn a partial directed acyclic graph (PDAG) from sample data. The variables correspond to the vertices. We use it to show the regulation relationships of each measured variable. The directed edge can reveal fairly strong causal relation, while the indirected edge only shows the correlativity between each variable.

Let Adjacencies(C, A) be the set of vertices adjacent to A in directed acyclic graph C. (In the algorithm, the graph C is continually updated, so Adjacencies(C,A) is constantly changing as the algorithm progresses.)

Algorithm 2 PC

Input: Measured data on the vertex set V.

Output: PDAG

1: Form the complete indirected graph C on the vertex set V

2: n=0

3: repeat

4: repeat

select an ordered pair of variables X and Y that are adjacent in C such that Adjacencies(C,X)\{Y} has cardinality greater than or equal to n, and a subset S of Adjacencies(C,X)\{Y} of cardinality n, and if X and Y are 8:d-separated given S delete edge X-Y from C and record S in Sepset(X,Y) and Sepset(Y,X);

until all ordered pairs of adjacent variables X and Y such that Adjacencies(C,X)\{Y} has cardinality greater than or equal to n and all subsets S of Adjacencies(C,X)\{Y} of cardinality n have been tested for d-separation;

5: n = n+1

6: Until for each ordered pair of adjacent vertices X, Y, Adjacencies(C,X)\{Y} is of cardinality less than n.

7: For each triple of vertices X, Y, Z such that the pair X, Y and the pair Y, Z are each adjacent in C but the pair X, Z are not adjacent in C, orient X-Y-Z as $X \to Y \leftarrow Z$ if and only if Y is not in Sepset(X,Z).

8:repeat

9: if $A \to B$, B and C are adjacent, A and C are not adjacent, and there is no arrowhead at B, then orient B-C as $B \to C$

10: if there is a directed path from A to B, and an edge between A and B, then orient A-B as $A \to B$

11: until no more edges can be oriented.

1.3 Incomplete Cholesky

A positive semidefinite matrix K can be factored as GG^T, where G is an $N \times N$ matrix. This factorization can be found via Cholesky decomposition. Our goal is to find a matrix $\tilde{G}$ of size $N \times M$, for small M, such that the difference $K - \tilde{G}\tilde{G}^T$ has norm less than a given value η. This algorithm is incomplete Cholesky decomposition. Incomplete Cholesky decomposition differs from standard Cholesky decomposition in that all pivots that are below a certain threshold are simply skipped. If M is the number of non-skipped pivots, then we obtain a lower triangular matrix $\tilde{G}$ with only M nonzero columns. Symmetric permutations of rows and columns are necessary during the factorization if we require the rank to be as small as possible. In that case, the stopping criterion involves the sum of remaining pivots. An algorithm for incomplete Cholesky decomposition is presented in Algorithm 2. The algorithm involves picking one column of K at a time, choosing the column to be added by greedily maximizing a lower bound on the reduction in the error of the approximation. After l steps, we have an approximation of the form $\tilde{K}_l = \tilde{G}\tilde{G}^T$, where G_l is $N \times l$. The ranking of the $N - l$ vectors that might be added in the following step is done through comparison of O(l) complexity. Moreover, the update of $\tilde{G}$ has a cost of O(lN), so that the overall complexity is $O(M^2N)$ (Bach et al., 2002).

Algorithm 3 Incomplete Cholesky

Input：$N \times N$ semidefinite positive matrix K precision parameter η

1: Initialization: i=1, $K' = K$, $P = I$, for $j \in [1, N]$, $G_{jj} = K_{jj}$

2: While $\sum_{j=i}^{N} G_{jj} > \eta$

3: Find best new element: $j^* = \mathrm{argmax}_{j \in [i,N]} G_{jj}$

4: Update permutation P:

5: set $P_{ii} = 0$, $P_{j^*j^*} = 0$ and $P_{i\,j^*} = 1$

6: Permute elements i and j* in K':

column $K'_{1:N,i} \leftrightarrow K'_{1:N,j^*}$

row $K'_{i,1:N} \leftrightarrow K'_{j^*,1:N}$

7: Update (due to new permutation) the already

calculated elements of G:$G_{i,1:i} \leftrightarrow G_{j^*,1:i}$

8: Set $G_{ii} = \sqrt{K'_{ii}}$

9: Calculate ith column of G: $G_{i+1:n,i} = \frac{1}{G_{ii}}(K'_{i+1:N,i} - \sum_{j=1}^{i-1} G_{i+1:N,j}G_{ij})$

10: Update only diagonal elements:

11: for $j \in [i+1, N], G_{jj} = K_{ii} - \sum_{k=1}^{i} G_{jk}^2$

12: $i \leftarrow i + 1$

Output: P, G and M = i-1